TP N°2: Régression Logistique

Dans ce travail pratique, nous allons développer un modèle de régression logistique pour réaliser une classification binaire. Nous utiliserons le jeu de données Pima, une ressource largement reconnue en apprentissage automatique. Ce jeu de données peut être utilisé pour prédire la probabilité de développer le diabète en fonction de facteurs tels que l'âge, le poids, le nombre de grossesses, etc.

- **Partie 1: Préparation du Corpus**

1. Téléchargez le jeu de données Pima (format CSV)
2. Chargez le jeu de données dans Python

```
data = pd.read_csv("your_pima_file_name.csv")
```

3. Affichez les 10 premières lignes pour comprendre la structure des données.

```
data....
```

4. Divisez le jeu de données en matrice de caractéristiques (X) et vecteur cible (y).

```
X = data... # Features
y = data... # Target
```

5. Ajoutez un terme d'interception (biais) à X et initialisez-le à 1.

```
X = ...
```

6. Quel est le but de ce code ? Expliquez chaque ligne :

```
def train_test_split(X, y, test_size=0.2, random_seed=42):
 np.random.seed(random_seed)
 indices = np.random.permutation(len(y))
 split_idx = int(len(y) * (1 - test_size))
 train_idx, test_idx = indices[:split_idx], indices[split_idx:]
 return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
```

[1]https://www.kaggle.com/api/v1/datasets/download/uciml/pima-indians-diabetes-database

- **Partie 2: Analyse exploratoire des données**

1. Calculez les statistiques de base (moyenne, médiane, écart-type) pour chaque caractéristique.

```
for column in data.columns:
 mean_value = ...
 median_value = ...
 std_dev_value = ...

 print(f"Feature: {column}")
 print(f"Mean: {mean_value}")
 print(f"Median: {median_value}")
 print(f"Standard Deviation: {std_dev_value}\n")
```

2. Visualisez les données à l'aide d'histogrammes et de diagrammes en boîte (boxplots).
3. Visualisez la distribution de la variable cible (Y).
4. Que fait ce code et que pouvons-nous apprendre de ce graphique ?

```
import seaborn as sns
corr_matrix = data.corr()
plt.figure(figsize=(10,10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.show()
```

- **Partie 3: Implementation of the Logistic Regression**

1. Définir des fonctions auxiliaires :
   - Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

   - Cost function (binary cross-entropy loss)

$$J(\vartheta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

   - Gradient computation (To avoid numerical instability from log(0),add a small value (e.g., $1e-15$) to $\hat{y}$ during computation)

$$\frac{\partial J(\vartheta)}{\partial \vartheta} = -\frac{1}{n}\sum_{i=1}^{n}\frac{y_i}{\hat{y}_i} - \frac{1-y_i}{1-\hat{y}_i}\, x_i = -\frac{1}{n}\sum_{i=1}^{n}\frac{y_i-\hat{y}_i}{(1-\hat{y}_i)\hat{y}_i}\, x_i$$

2. Implémentez la fonction de descente de gradient pour mettre à jour θ. Le code ci-dessous fournit une structure de base de cette fonction. Complétez-la avec les instructions nécessaires pour obtenir le résultat souhaité.

```
def gradient_descent(X, y, learning_rate=0.01, epochs=1000):
 # Initialize parameters
 theta = ... # Initial weights
 theta_history = error_history = [] # To store MSE values

 for epoch in range(epochs):
  # Linear model
  z = ...

  # Predicted probabilities
  y_pred = ...

  # Compute gradients
  gradient = ...

  # Update weights (theta)
  ...

  # Compute the cost function
  current_error = ...

  # Save current theta and error for tracking
  ...
  ...
return theta_history, error_history, theta
```

3. Initialisez les paramètres : taux d'apprentissage, nombre d'époques et seuil de convergence (ce paramètre est optionnel)

```
# Initialize Parameters
learning_rate = ...
epochs = ...
convergence_threshold = ... #optional parameter
```

4. Entraînez le modèle pendant un nombre fixe d'itérations ou jusqu'à ce que la convergence soit atteinte. Modifiez la fonction de descente de gradient pour intégrer un critère de convergence.

```
# Train the model
theta_history, error_history, final_theta = ...
```

5. Prédire les résultats. La fonction `predict` donnée calcule les prédictions pour la régression logistique en utilisant un seuil (complétez le code) :

```
# Predict function
def predict(X, theta, threshold=0.5):
 probabilities = ... #implement a Step 1
 predictions = ... #implement a Step 2
 return predictions
```

(a) Étape 1 : Calculez les probabilités en utilisant la fonction logistique (sigmoïde) appliquée à la combinaison linéaire des entrées et des poids :

$$P(y = 1 \mid \mathbf{X}) = \sigma(\mathbf{X} \cdot \mathbf{w}) = \frac{1}{1 + e^{-(\mathbf{X} \cdot \mathbf{w})}}$$

(b) Étape 2: Convertissez les probabilités en résultats binaires en utilisant un seuil (par exemple, 0,5).

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1 \mid \mathbf{X}) \geq \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

- **Partie 4: Visualisation de la perte (loss):**

1. Tracez la perte (loss) pendant l'entraînement au cours des époques.

```
plt.figure(figsize=(10, 6))
...
plt.show()
```

Que pouvez-vous observer à partir de ce graphique ? Suggérez une modification pour mieux visualiser l'erreur d'apprentissage.

2. Prétraitez vos données en utilisant la technique de normalisation décrite ci-dessous.

$$X^{\text{normalized}} = \frac{X - \mu}{\sigma}$$

Où μ et σ sont des vecteurs contenant respectivement les moyennes et les écarts-types de chaque caractéristique de X.
Ajoutez cette ligne de code au début de votre code (après la Question 4 de la section Préparation du jeu de données).

```
# Normalize the features
X = (X - ....) / ....
```